

Introduction

An **Exception** is an **abnormal event** that occurs during program execution and disrupts the normal flow of the program.

If an exception is not handled, the program terminates abnormally.

Exception Handling in Java is a mechanism used to handle both compile-time (checked) and runtime (unchecked) exceptions, allowing a program to continue execution smoothly even in the presence of errors.

- Handles abnormal conditions that occur during program execution.
 - Helps maintain program stability by preventing unexpected termination.
-

Exception Handling Keywords

1. `try`

The `try` block contains the code that may generate an exception. It must be followed by at least one `catch` or `finally` block.

2. `catch`

The `catch` block is used to handle an exception thrown by the `try` block and prevent abnormal program termination.

3. `finally`

The `finally` block contains code that is always executed, whether an exception occurs or not.

4. `throw`

The `throw` keyword is used to explicitly throw an exception object from a method or block of code.

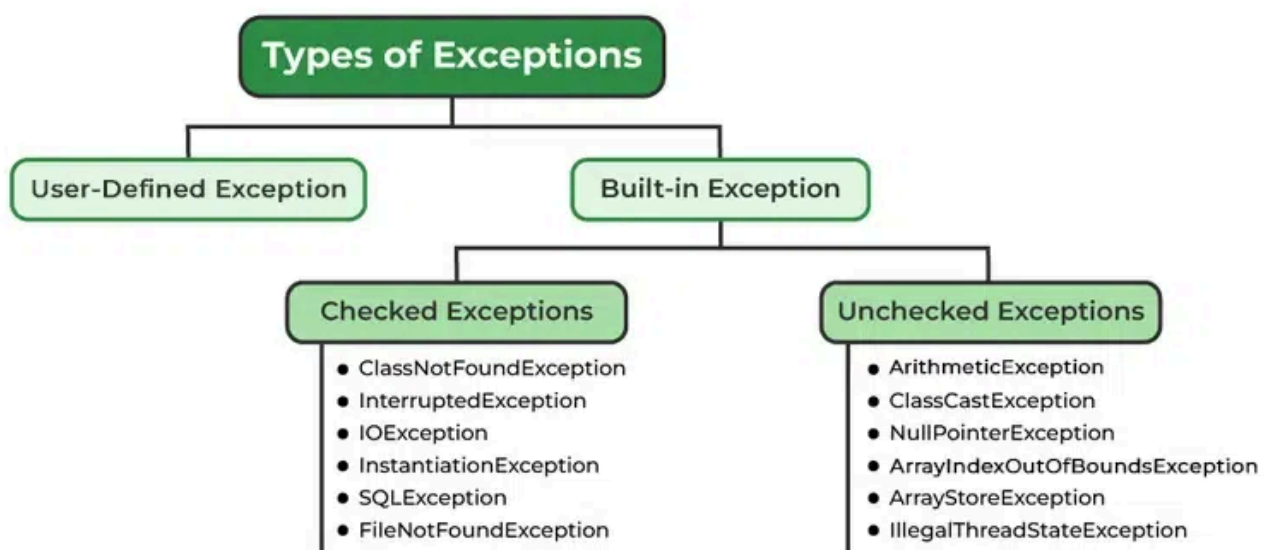
5. throws

The `throws` keyword is used in a method declaration to specify the exceptions that the method may throw.

Difference Between `throw` and `throws`

<code>throw</code>	<code>throws</code>
Used to explicitly throw an exception.	Used to declare exceptions in a method signature.
Used inside the method body.	Used with the method declaration.
Throws a single exception object.	Can declare one or more exceptions.

Types of Exceptions



Built-in Exception

Built-in Exception are pre-defined exception classes provided by Java to handle common errors during program execution. There are two type of built-in exception in java.

Checked Exception

Checked exceptions are checked by the **compiler** during compilation. They must be handled using `try-catch` or declared using `throws`.

Examples

- IOException
- FileNotFoundException
- SQLException

Example

```
import java.io.*;

class Demo{
    public static void main(String args[]) {
        FileReader f = new FileReader("abc.txt");
    }
}
```

The compiler gives an error because the exception is not handled.

Unchecked Exception

Definition

Unchecked exceptions occur during **runtime**. The compiler does not force the programmer to handle them.

Examples

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException

Example

```
int a = 10;
int b = 0;

System.out.println(a / b);
```

Output

ArithmeticException

User-Defined Exception

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called "user-defined Exceptions".

```
class AgeException extends Exception
{
    AgeException(String msg)
    {
        super(msg);
    }
}

class Demo
{
    public static void main(String args[])
    {
        int age = 15;

        try
        {
            if(age < 18)
            {
                throw new AgeException("Not Eligible");
            }

            System.out.println("Eligible");
        }
    }
}
```

```
        catch(AgeException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

How Does JVM Handle an Exception?

When an Exception occurs, the JVM creates an exception object containing the error name, description, and program state. Throwing an exception means creating an exception object and transferring control to the nearest appropriate exception handler using the throw keyword. There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called call stack. Now the following procedure will happen:

- The run-time system searches the call stack for an exception handler
- It starts searching from the method where the exception occurred and proceeds backward through the call stack.
- If a handler is found, the exception is passed to it.
- If no handler is found, the default exception handler terminates the program and prints the stack trace.

Questions

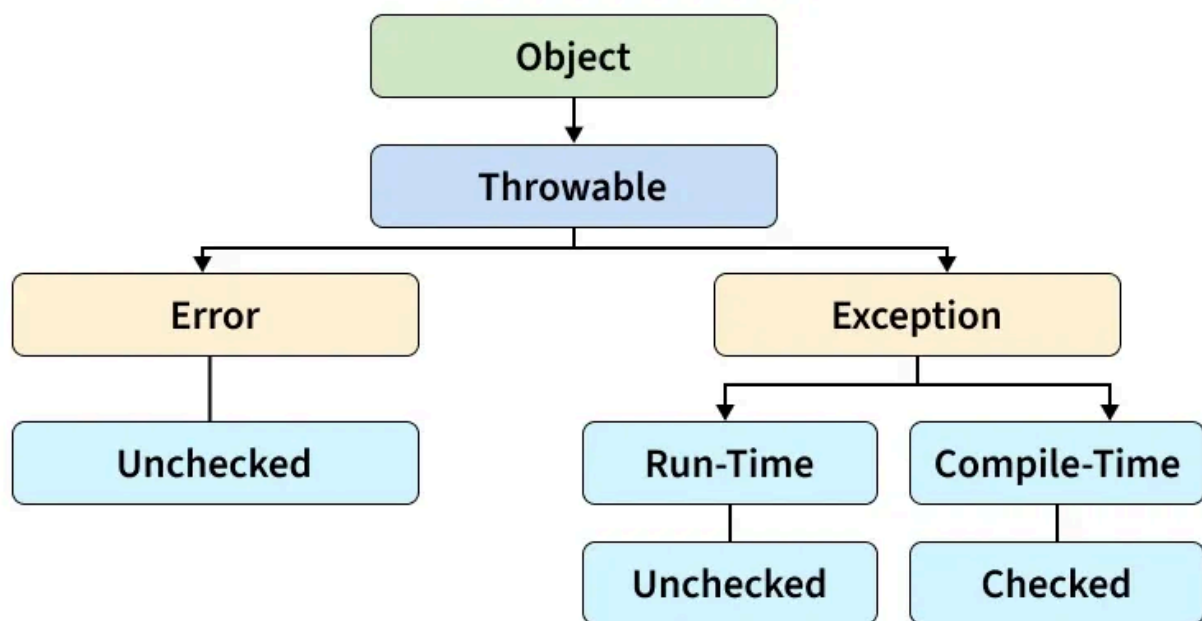
1. Explain the exception class hierarchy and write a program demonstrating multiple catch blocks along with a user-defined exception.

In Java, all exceptions are derived from the **Throwable** class. The `Throwable` class has two main subclasses:

1. **Error** – Represents serious system-level errors that are generally not handled by the programmer.
2. **Exception** – Represents exceptions that occur during program execution and can be handled using exception handling mechanisms.

The `Exception` class is further divided into:

- **Checked Exceptions** – Checked at compile time (e.g., `IOException`, `SQLException`).
- **Unchecked Exceptions** – Occur at runtime and inherit from `RuntimeException` (e.g., `ArithmeticException`, `NullPointerException`).



```
class InvalidAgeException extends Error {
    InvalidAgeException(String msg) {
        super();
    }
}

public class Main {
    public static void main(String[] args) {
        int age = 16;

        try {
            int a = 10 / 0;

            int arr[] = new int[5];
```

```

        arr[7] = 20;

        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or
above.");
        }

        System.err.println(a);
    }

    catch (ArithmeticException e) {
        System.out.println("Arithmetic Exception: " +
e.getMessage());
    }

    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array Index Exception: " +
e.getMessage());
    }

    catch (InvalidAgeException e) {
        System.out.println("User Defined Exception: " +
e.getMessage());
    }

    finally {
        System.out.println("Program execution completed.");
    }
}
}

```

2. Write a Java program to accept two integers from the user and perform division. Handle `ArithmeticException` and `InputMismatchException` using multiple `catch` blocks. Display a message using the `finally` block.

```

import java.util.InputMismatchException;
import java.util.Scanner;

public class Main {

```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try{
        System.out.print("Enter first value: ");
        int v1 = sc.nextInt();

        System.out.print("Enter second value: ");
        int v2 = sc.nextInt();

        int answer = v1 / v2;

        System.out.println(answer);
    }

    catch (ArithmeticException e){
        System.out.println("Arithmetic Exception: " +
e.getMessage());
    }

    catch(InputMismatchException e)
    {
        System.out.println("Error: Please enter integers
only.");
    }

    finally{
        sc.close();
    }
}
}
```